

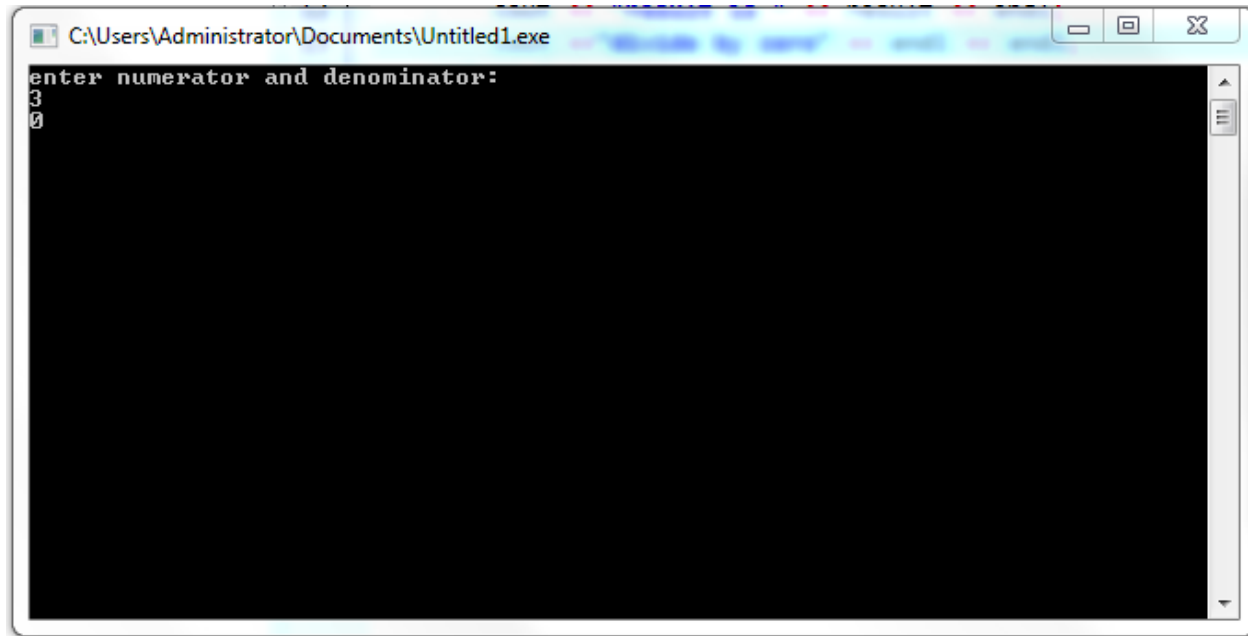
Object-oriented Programming

Week 13 | Lecture 2

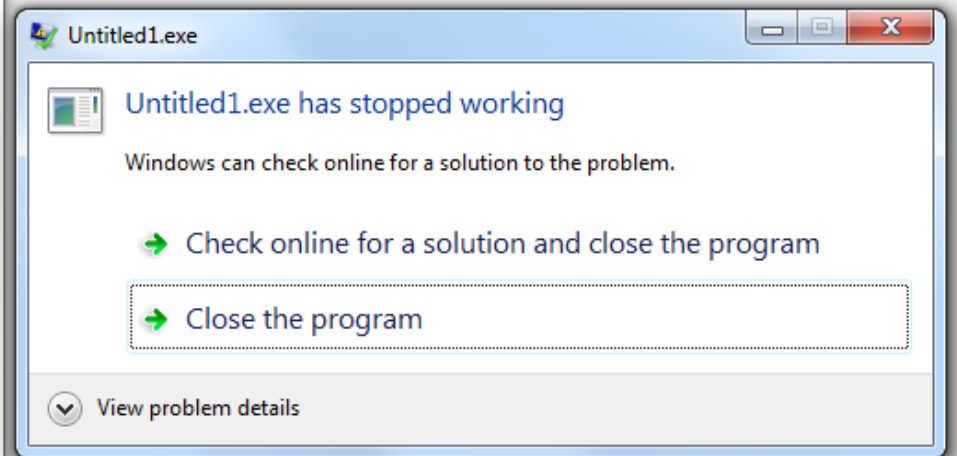
Output?

```
#include <iostream>
using namespace std;
double zeroDivision(int x, int y) {
    return (x / y);}
int main() {
    int numerator;
    int denominator;
    double result;
    cout << "enter numerator and denominator: " << endl;
    cin>>numerator>>denominator; // 3 and 0
        result = zeroDivision(numerator, denominator);
        cout << "result is " << result << endl;
        cout <<"divide by zero" << endl << endl;
    return 0;}
```


Output?



```
C:\Users\Administrator\Documents\Untitled1.exe
enter numerator and denominator:
3
0
```



Exceptions

- Exception handling in C++ provides you with a way of handling unexpected circumstances like runtime errors. So whenever an unexpected circumstance occurs, the program control is transferred to special functions known as handlers.
- An *exception is an abnormal* condition that arises in a code sequence at run time
- In other words, an exception is a run-time error

Handlers

- Exceptions provide a way to transfer control from one part of a program to another. C++ exception handling is built upon three keywords:
- **try,**
- **catch,** and
- **throw.**

Handlers

- **throw**- when a program encounters a problem, it throws an exception. The throw keyword helps the program perform the throw.
- **catch**- a program uses an exception handler to catch an exception. It is added to the section of a program where you need to handle the problem. It's done using the catch keyword.
- **try**- the try block identifies the code block for which certain exceptions will be activated. It should be followed by one/more catch blocks.

Catching Exceptions

- The **catch** block following the **try** block catches any exception. You can specify what type of exception you want to catch and this is determined by the exception declaration that appears in parentheses following the keyword catch.
- try {
- // protected code
- } catch(ExceptionName exception1) {
- // code to handle ExceptionName exception
- }

Throwing Exceptions

- Exceptions can be thrown anywhere within a code block using **throw** statement. The operand of the throw statement determines a type for the exception and can be any expression and the type of the result of the expression determines the type of exception thrown.
- ```
double division(int a, int b) {
 if(b == 0) {
 throw "Division by zero condition!"; //throw b;
 }
 return (a/b);
}
```

# try

- A block of code which may cause an exception is typically placed inside the try block. It's followed by one or more catch blocks. If an exception occurs, it is thrown from the try block.
- `try {`
- `// protected code`
- `}`

# throw

- Possible to declare throw keyword anywhere in the try block. throw keyword trigger/execute a matched type of the catch block. The try block exists When a throw statement is reached.

# Catching Exceptions

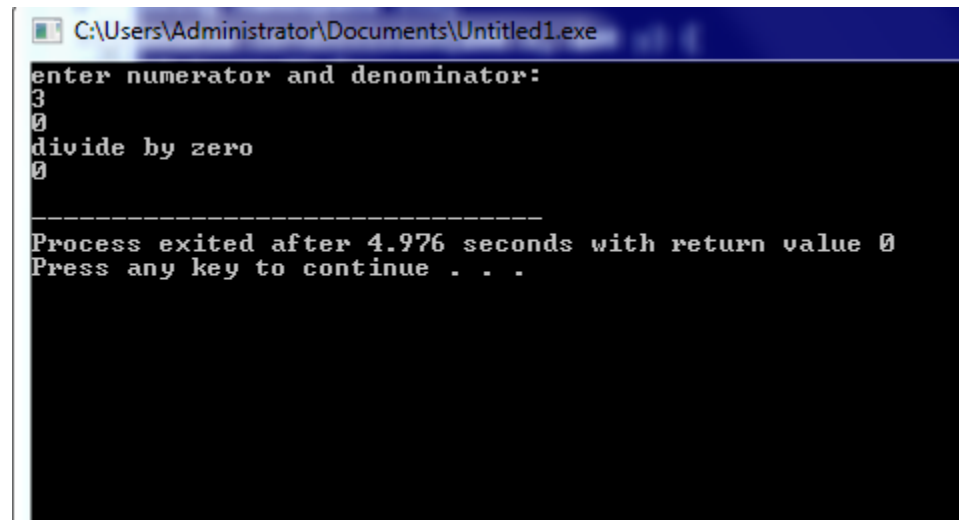
- The ExceptionName is the name of the exception to be caught.
- The exception1 are your defined names for referring to the exceptions.

# Example

```
#include <iostream>
using namespace std;
double zeroDivision(int x, int y) {
 if (y == 0) {
 throw y;}
 return (x / y);}
int main() {
 int numerator;
 int denominator;
 double result;
 cout << "enter numerator and denominator: " << endl;
 cin>>numerator>>denominator;

 try {
 result = zeroDivision(numerator, denominator);
 cout << "result is " << result << endl; }
 catch (int ex) {
 cout <<"divide by zero" << endl << ex << endl; }
 return 0;}
```

# Example



```
C:\Users\Administrator\Documents\Untitled1.exe
enter numerator and denominator:
3
0
divide by zero
0

Process exited after 4.976 seconds with return value 0
Press any key to continue . . .
```

The image shows a Windows command prompt window titled "C:\Users\Administrator\Documents\Untitled1.exe". The prompt asks for a numerator and denominator. The user enters "3" for the numerator and "0" for the denominator. This results in a "divide by zero" error, which is displayed in red text. The error code "0" is shown on the next line. A horizontal line separates this from the final message: "Process exited after 4.976 seconds with return value 0". The prompt then says "Press any key to continue . . .".

```
#include <iostream>
using namespace std;
int main() {
 double balance = 5000.0;
 double withdraw;
 cout << "Enter amount to withdraw: ";
 cin >> withdraw;
 try {
 if (withdraw > balance)
 throw "Insufficient Balance!";
 balance -= withdraw;
 cout << "Withdrawal successful. New balance: " << balance << endl; }
 catch (const char* msg) {
 cout << "Exception: " << msg << endl; }

 return 0;
}
```

# Exception Handling Checklist

- Try block has the main code, which code may throw an exception.
- Multiple throws are possible to declare in try block based different situations.
- The catch block contains exception handling code that is executed when thrown by the Try Block.
- Possible to declare one or more catch block for different type of exception
- No code can be between try block and the first catch block.
- Catch blocks declare directly after the try block.

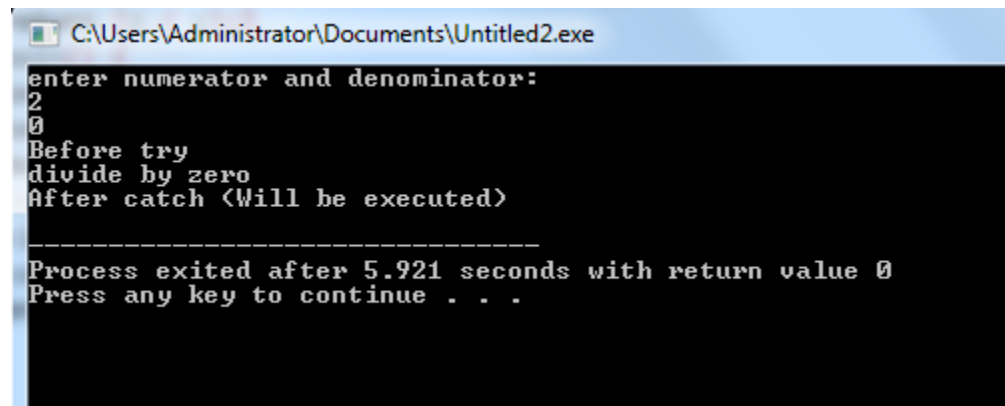
# flow of execution of try/catch blocks.

- `#include <iostream>`
- `#include <exception>`
- `using namespace std;`
- `double zeroDivision(int x, int y) {`
- `if (y == 0) {`
- `throw y;`
- `cout << "After throw (Never executed) \n";}`
- `return (x / y);}`
- `int main() {`
- `int numerator;`
- `int denominator;`
- `double result;`
- `cout << "enter numerator and denominator: " << endl;`
- `cin>>numerator>>denominator;`
- `cout << "Before try \n";`

# flow of execution of try/catch blocks.

- try {
- result = zeroDivision(numerator, denominator);
- cout << "result is " << result << endl; }
- catch (int e) {
- cout <<"divide by zero" << endl;       }
- cout << "After catch (Will be executed) \n";
- return 0;}

# flow of execution of try/catch blocks.



```
C:\Users\Administrator\Documents\Untitled2.exe
enter numerator and denominator:
2
0
Before try
divide by zero
After catch <Will be executed>

Process exited after 5.921 seconds with return value 0
Press any key to continue . . .
```

# Without catch

- `#include <iostream>`
- `#include <exception>`
- `using namespace std;`
- `double zeroDivision(int x, int y) {`
- `if (y == 0) {`
- `throw y;}`
- `return (x / y);}`
- `int main() {`
- `int numerator;`
- `int denominator;`
- `double result;`
- `cout << "enter numerator and denominator: " << endl;`
- `cin >> numerator >> denominator;`
- 
- `try {`
- `result = zeroDivision(numerator, denominator);`
- `cout << "result is " << result << endl; }`
- 
- `return 0;}`

# Without catch

In function 'int main()':

```
[Error] expected 'catch' before numeric constant
```

# Multiple catch Clauses

- In some cases, more than one exception could be raised by a single piece of code
- To handle this type of situation, you can specify two or more catch clauses, each catching a different type of exception
- After one catch statement executes, the others are bypassed

# Multiple catch Clauses

- You can list down multiple **catch** statements to catch different type of exceptions in case your **try** block raises more than one exception in different situations.
- `try { // protected code }`
- `catch( ExceptionName e1 )`
- `{ // catch block }`
- `catch( ExceptionName e2 ) { // catch block }`
- `catch( ExceptionName eN ) { // catch block }`

# Multiple catch Clauses

- `#include <iostream>`
- `#include <exception>`
- `using namespace std;`
- `double zeroDivision(int x, int y) {`
- `if (y == 0) {`
- `throw y;`
- `}`
- `return (x / y);`
- `}`
- `int main() {`
- `int numerator;`
- `int denominator;`
- `double result;`
- `cout << "enter numerator and denominator: " << endl;`
- `cin >> numerator >> denominator;`
- `try {`
- `result = zeroDivision(numerator, denominator);`
- `cout << "result is " << result << endl;`
- `}`

# Multiple catch Clauses

- `catch (int e) {`
- `cout <<"divide by zero" << endl << e << endl;        }`
- `catch (int e) {`
- `cout <<"another exception" << endl << e << endl;  }`
- `catch (int e) {`
- `cout <<"another exception" << endl << e << endl;  }`
- `return 0;}`

# catch all block

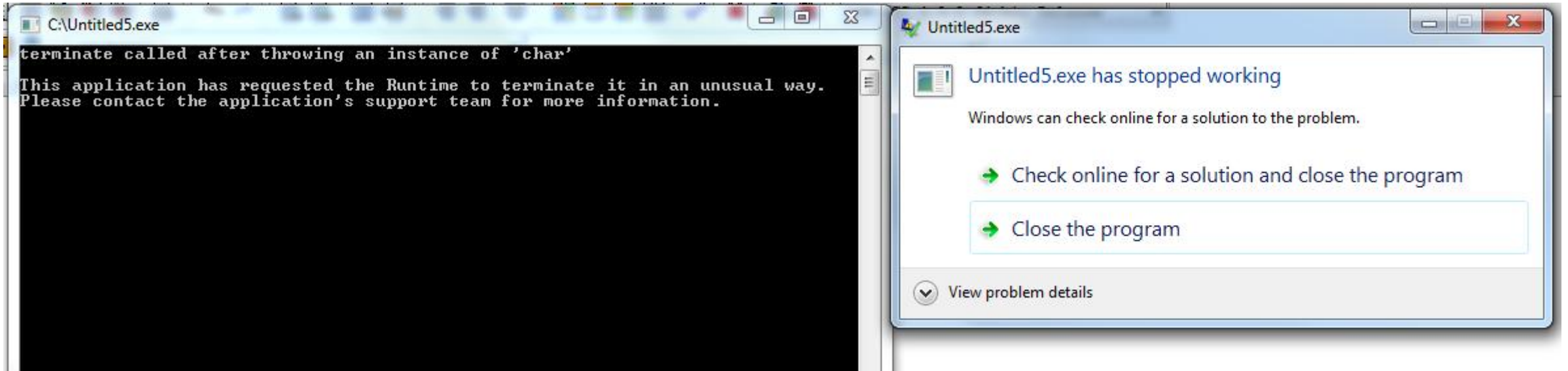
- If you want to specify that a catch block should handle any type of exception that is thrown in a try block, you must put an ellipsis, ..., between the parentheses enclosing the exception declaration as follows –
- `try { // protected code }`
- `catch(...)`
- `{ // code to handle any exception }`

# catch all block

- `#include <iostream>`
- `using namespace std;`
- `double zeroDivision(int x, int y) {`
- `if (y == 0) {`
- `throw y;}`
- `return (x / y);}`
- `int main() {`
- `int numerator;`
- `int denominator;`
- `double result;`
- `cout << "enter numerator and denominator: " << endl;`
- `cin >> numerator >> denominator;`
- `try {`
- `result = zeroDivision(numerator, denominator);`
- `cout << "result is " << result << endl; }`
- `catch(...){`
- `cout << "divide by zero" << endl << endl; }`
- `return 0;}`

- If an exception is thrown and not caught anywhere, the program terminates abnormally.
- `#include <iostream>`
- `using namespace std;`
- 
- `int main(){`
- `try {`
- `throw 'a'; }`
- `catch (int x) {`
- `cout << "Caught "; }`
- `return 0;}`

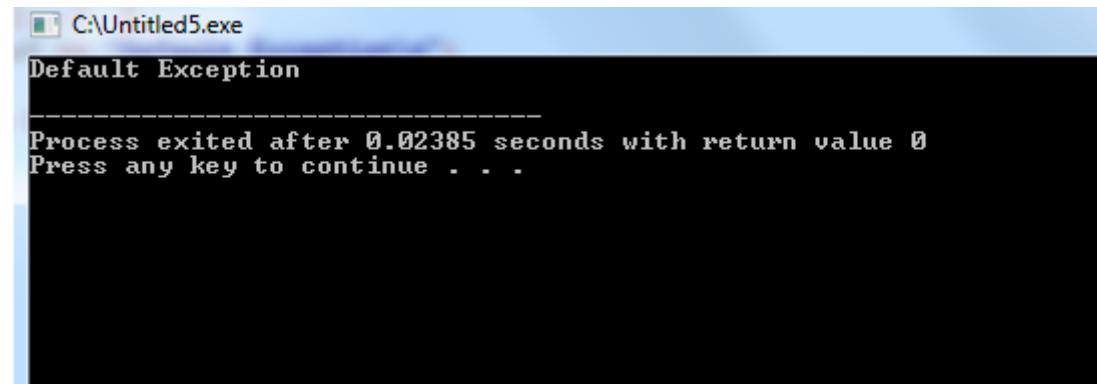
- Implicit type conversion doesn't happen for primitive types. For example, in the following program 'a' is not implicitly converted to int



# Correct version

- `#include <iostream>`
- `using namespace std;`
- 
- `int main()`
- `{`
- `try {`
- `throw 'a';`
- `}`
- `catch (int x) {`
- `cout << "Caught " << x;`
- `}`
- `catch (...) {`
- `cout << "Default Exception\n";`
- `}`
- `return 0;`
- `}`

# output



```
C:\Untitled5.exe
Default Exception

Process exited after 0.02385 seconds with return value 0
Press any key to continue . . .
```

# Purpose of rethrow;

Sometimes you want to:

1. Catch an exception temporarily,
2. Do something small (like logging, cleaning up),
3. Then throw it again so someone else can handle it further up the call stack.

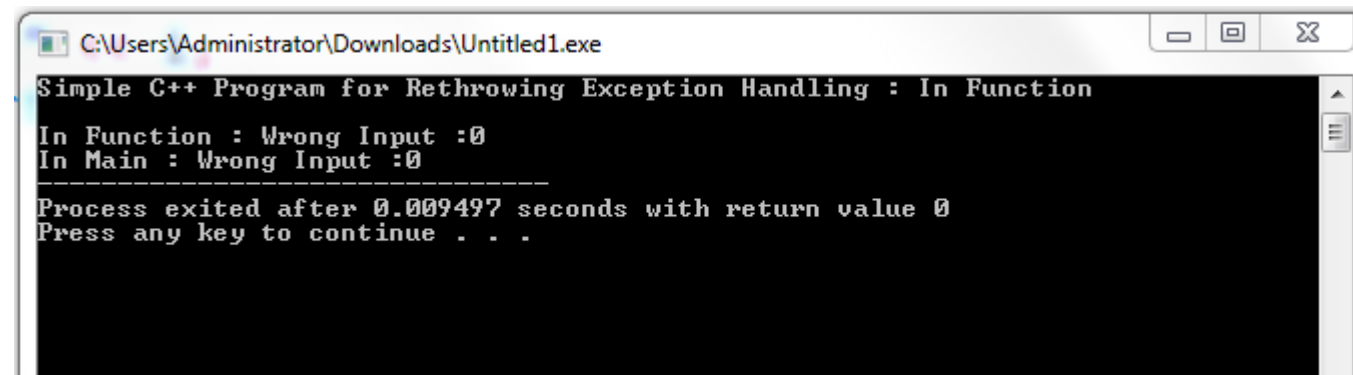
Example: You log the error here, but the main function or some bigger handler should deal with it.

`throw;` is used to "pass forward" an already caught exception to another catch higher up.

# Simple C++ Program for Rethrowing Exception Handling in Function

```
void exceptionFunction() {
 try {
throw 0;
 } catch (int i) {
 cout << "\nIn Function : Wrong Input :" << i;
 //re throw exception : out of the function
 throw; }
}

int main() {
 int var = 0;
 cout << "Simple C++ Program for Rethrowing Exception Handling : In Function\n";
 try {
exceptionFunction(); }// catch block
 catch (int ex) {
 // Code executed when exception caught
 cout << "\nIn Main : Wrong Input :" << ex; }
 return 0;}
```



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\Administrator\Downloads\Untitled1.exe" and standard window controls (minimize, maximize, close). The command prompt has a black background with white text. The output of the program is as follows:

```
Simple C++ Program for Rethrowing Exception Handling : In Function
In Function : Wrong Input :0
In Main : Wrong Input :0

Process exited after 0.009497 seconds with return value 0
Press any key to continue . . .
```

- `#include <iostream>`
- `using namespace std;`
- `void riskyFunction() {`
- `throw "Something bad happened!";`
- `}`
- `int main() {`
- `try {`
- `try {`
- `riskyFunction();`
- `}`
- `catch (const char* msg) {`
- `cout << "Logging inside inner catch: " << msg << endl;`
- `throw; // Rethrow the same exception`
- `} }`
- `catch (const char* msg) {`
- `cout << "Handling exception in outer catch: " << msg << endl; }`
- `return 0;`
- `}`

## Rule

`throw;` must be inside a `catch` block

It rethrows the *currently caught* exception

Good for layered error handling

## Explanation

Otherwise, it will crash (compiler/runtime error).

No new object, same one!

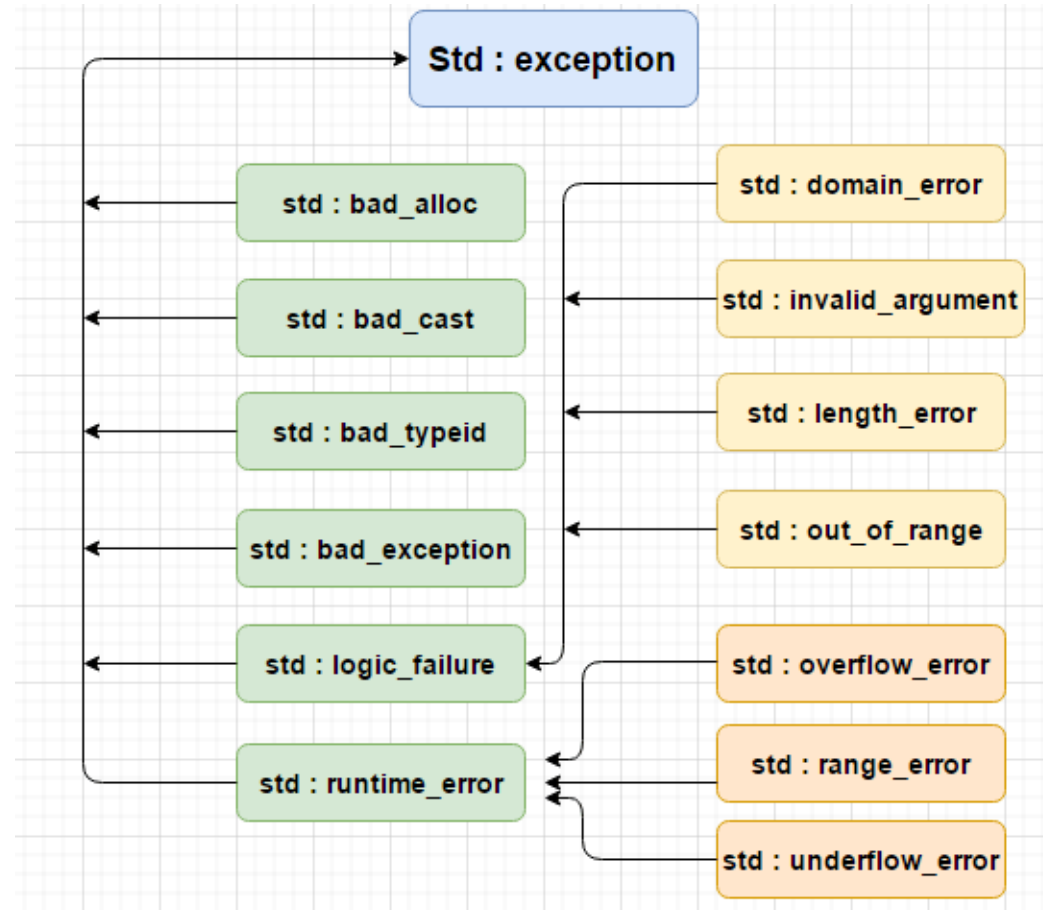
You can log in one place and fully handle elsewhere.

# Output?

- `#include <iostream>`
- `using namespace std;`
- `int main() {`
- `try {`
- `throw "Initial exception!";   }`
- `catch (const char* msg) {`
- `cout << "Caught inside catch: " << msg << endl;`
- `// Rethrow the same exception`
- `throw;`
- `}`
- `// This will never be executed because the program terminates`
- `cout << "After catch." << endl;`
- 
- `return 0;`
- `}`

- `#include <iostream>`
- `using namespace std;`
- `int main() {`
- `try {`
- `throw "Initial exception!"; }`
- `catch (const char* msg) {`
- `cout << "Caught inside catch: " << msg << endl;`
- `// Rethrow the same exception`
- `throw; }`
- `catch (const char* msg) {`
- `cout << "Caught outer catch: " << msg << endl;`
- `}`
- `cout << "After catch." << endl;`
- 
- `return 0;`
- `}`

# C++ Standard Exceptions



# C++ Standard Exceptions

- C++ provides a list of standard exceptions defined in `<exception>` which we can use in our programs

# C++ Standard Exceptions

| Exception                          | Description                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>std::exception</code>        | An exception and parent class of all the standard C++ exceptions.                                                                                 |
| <code>std::bad_alloc</code>        | This can be thrown by <code>new</code> .                                                                                                          |
| <code>std::bad_cast</code>         | This can be thrown by <code>dynamic_cast</code> .                                                                                                 |
| <code>std::bad_exception</code>    | This is useful device to handle unexpected exceptions in a C++ program                                                                            |
| <code>std::bad_typeid</code>       | This can be thrown by <code>typeid</code> .                                                                                                       |
| <code>std::logic_error</code>      | An exception that theoretically can be detected by reading the code.                                                                              |
| <code>std::domain_error</code>     | This is an exception thrown when a mathematically invalid domain is used                                                                          |
| <code>std::invalid_argument</code> | This is thrown due to invalid arguments.                                                                                                          |
| <code>std::length_error</code>     | This is thrown when a too big <code>std::string</code> is created                                                                                 |
| <code>std::out_of_range</code>     | This can be thrown by the <code>at</code> method from for example a <code>std::vector</code> and <code>std::bitset&lt;&gt;::operator[]()</code> . |
| <code>std::runtime_error</code>    | An exception that theoretically can not be detected by reading the code.                                                                          |
| <code>std::overflow_error</code>   | This is thrown if a mathematical overflow occurs.                                                                                                 |
| <code>std::range_error</code>      | This is occurred when you try to store a value which is out of range.                                                                             |
| <code>std::underflow_error</code>  | This is thrown if a mathematical underflow occurs.                                                                                                |

# User-Defined Exceptions

- The C++ `std::exception` class allows us to define objects that can be thrown as exceptions. This class has been defined in the `<exception>` header. The class provides us with a virtual member function named `what`.
- This function returns a null-terminated character sequence of type `char *`. We can overwrite it in derived classes to have an exception description.

- `#include <iostream>`
- `using namespace std;`
- `class MyException {`
- `public:`
- `const char* what() const {`
- `return "Something went wrong!";`
- `}};`
- `int main() {`
- `try {`
- `MyException ex;`
- `throw ex; // Throwing an object`
- `}`
- `catch (MyException& e) { // Catching by reference (good practice)`
- `cout << "Caught an exception: " << e.what() << endl;`
- `}`
- `return 0;`
- `}`

# User-Defined Exceptions

```
#include <iostream>
#include <exception>
using namespace std;
class newException : public exception{
 virtual const char* what() const throw()//We can overwrite the what() function of the exception header file to define our
exceptions.
{
 return "newException occurred"; }
} newex;
int main() {
try {
 throw newex;
} catch (exception& ex) {
 cout << ex.what() << '\n';
}
return 0; }
```